TALLER DE PROGRAMACIÓN / CLASE #02

Guía Teórica -

1. ¿Qué es una función?
2. Funciones sin parámetros
3. Representación de una función
4. Representación de una función
5. Abstracción
6. Ejemplos teórico-prácticos integradores
99. Ejercitación propuesta

1. ¿Qué es una función?

Una función es un bloque de código con nombre propio que agrupa una secuencia de instrucciones para realizar una tarea específica. Una vez definida, se puede invocar (llamar) tantas veces como sea necesario, sin tener que reescribir el código.

¿Para qué sirven?

- Reutilización: escribís el código una vez y lo usás muchas veces.
- Organización: dividen un programa grande en partes más pequeñas y manejables.
- Claridad: un buen nombre de función explica qué hace, sin necesidad de leer su interior.
- Mantenimiento: si hay que corregir algo, se corrige en un solo lugar.

Pensá en una función como una receta de cocina: tiene un nombre ("hacer café"), una serie de pasos internos, y quien la usa no necesita saber cómo se hace el café en detalle, solo necesita "pedir" que se haga.

Ejemplo. -

```
def saludar():
    print("¡Hola! Bienvenido al Taller de programacion.")

# Para que la función se ejecute, hay que llamarla:
saludar()
```

Si solo se define la función pero nunca se la llama, el mensaje nunca se muestra en pantalla. Definir no es lo mismo que ejecutar.

2. Funciones sin parámetros

Una función sin parámetros es aquella que no recibe ningún dato desde afuera para trabajar. Todo lo que necesita ya está definido dentro de su propio cuerpo (o son datos fijos, o los pide con input(), o usa variables globales).

Sintaxis general

```
def nombre_de_la_funcion():
    # cuerpo de la función (instrucciones)
    # ...
```

Partes clave:

- def: palabra reservada que indica que se está definiendo una función.
- nombre_de_la_funcion: identificador (sigue las mismas reglas que las variables).
- (): paréntesis vacíos, porque no recibe parámetros.
- : dos puntos, obligatorios.
- Cuerpo indentado (4 espacios habitualmente).

Ejemplo: función que no devuelve nada (usa print)

```
def mostrar_menu():
    print("1. Iniciar sesión")
    print("2. Registrarse")
    print("3. Salir")

mostrar_menu()
```

Ejemplo: función que devuelve un valor (usa return)

```
def obtener_pi():
    return 3.14159

valor = obtener_pi()
print(valor)
```

print vs return: una distinción fundamental

Es uno de los errores conceptuales más comunes al empezar.

	print()	**return**
¿Qué hace?	Muestra algo en pantalla	Entrega (devuelve) un valor a quien llamó a la función
¿Se puede reutilizar el resultado?	No, solo se ve, no se guarda	Sí, se puede guardar en una variable
¿Detiene la función?	No	Sí, corta la ejecución en ese punto
¿Cuántas veces se puede usar?	Las que se quieran	Al ejecutarse una, la función termina ahí

```
def con_print():
    print("Hola")
```

```
def con_return():
    return "Hola"

a = con_print()   # Se imprime "Hola" en pantalla, pero a vale None
b = con_return()  # No se imprime nada en pantalla, pero b vale "Hola"

print(a)  # None
print(b)  # Hola
```

Funciones que combinan varias instrucciones

```
def informe_bienvenida():
    print("=== Sistema de Gestión ===")
    print("Versión 1.0")
    print("Todos los derechos reservados")

informe_bienvenida()
```

Funciones que usan input() (piden datos al usuario, no parámetros)

Ojo: pedir datos con input() no es lo mismo que recibir parámetros. Los parámetros se verán más adelante; acá el dato lo pide la propia función al usuario en el momento de ejecutarse.

```
def pedir_nombre():
    nombre = input("¿Cómo te llamás? ")
    print(f"Encantado de conocerte, {nombre}")

pedir_nombre()
```

3. Representación de una función

Anatomía completa

```
def nombre_funcion():      # <- Encabezado (header)
    """Docstring opcional: explica qué hace la función."""
    instruccion_1           # <- Cuerpo
    instruccion_2           # <- Cuerpo
    return resultado        # <- Salida (opcional)

nombre_funcion()            # <- Llamado / invocación
```

4. Abstracción

La abstracción es el principio que permite usar una función sin necesidad de conocer los detalles de su implementación interna. Solo hace falta saber:

- Qué hace (su propósito).
- Cómo se llama (su nombre).
- Qué devuelve o qué efecto produce (su resultado, si lo tiene).

No hace falta saber cómo lo hace por dentro.

Ejemplo cotidiano

Cuando usás len("hola") en Python, no necesitás saber cómo Python cuenta los caracteres internamente. Solo sabés que te va a devolver la longitud del texto. Eso es abstracción: ocultar la complejidad detrás de una interfaz simple.

Aplicándolo a funciones propias

```
def calcular_area_circulo_unitario():
    pi = 3.14159
    radio = 1
    area = pi * radio ** 2
    return area

# Quien usa la función no necesita saber la fórmula,
# solo necesita saber que "calcular_area_circulo_unitario"
# le devuelve el área.
resultado = calcular_area_circulo_unitario()
print(resultado)
```

Niveles de abstracción

```
Nivel 1 (uso):
resultado = calcular_area_circulo_unitario()
(no necesito ver esto para usarla)

Nivel 2 (implementación):
pi = 3.14159
radio = 1
area = pi * radio ** 2
return area
```

¿Por qué es importante en programación?

- Permite trabajar en equipo: una persona puede usar una función que escribió otra sin leer todo su código.
- Permite cambiar la implementación sin romper el resto del programa, siempre que el "contrato" (qué recibe y qué devuelve) se mantenga.
- Reduce la carga mental: no hay que pensar en todo el sistema al mismo tiempo, sino en partes pequeñas.

5. Ejemplos teórico-prácticos integradores

Ejemplo 1: Sistema de bienvenida

```
def linea_separadora():
    print("-" * 30)
```

```
def titulo():
    print("SISTEMA DE INSCRIPCIÓN")

def instrucciones():
    print("Complete los siguientes pasos:")
    print("1. Ingrese sus datos")
    print("2. Confirme la inscripción")

def mostrar_pantalla_inicial():
    linea_separadora()
    titulo()
    linea_separadora()
    instrucciones()
    linea_separadora()

mostrar_pantalla_inicial()
```

Concepto integrado: composición de funciones (una función que llama a otras), abstracción (cada función oculta su detalle) y organización del código.

Ejemplo 2: Cálculo con retorno de valores

```
def obtener_iva():
    return 0.21

def calcular_recargo_fijo():
    precio_base = 1000
    iva = obtener_iva()
    return precio_base * iva

recargo = calcular_recargo_fijo()
print(f"El recargo de IVA es: ${recargo}")
```

Concepto integrado: una función puede usar el resultado de otra función (obtener_iva() es usada dentro de calcular_recargo_fijo()), reforzando la idea de abstracción y reutilización.

Ejemplo 3: Menú interactivo con input()

```
def mostrar_menu():
    print("=== MENÚ ===")
    print("1. Ver saldo")
    print("2. Ver movimientos")
    print("3. Salir")

def pedir_opcion():
    opcion = input("Elija una opción: ")
    return opcion

def procesar():
    mostrar_menu()
    opcion = pedir_opcion()
```

```
    print(f"Usted eligió la opción {opcion}")

procesar()
```

Concepto integrado: separación de responsabilidades — cada función hace una sola cosa (mostrar, pedir, procesar), lo cual es una buena práctica de diseño.

Ejemplo 4: Comparación print vs return en un mismo flujo

```
def version_con_print():
    print("Procesando datos...")
    print("Resultado: 42")

def version_con_return():
    print("Procesando datos...")
    return 42

version_con_print()            # Muestra ambos mensajes, pero no se puede reutilizar "42"
valor = version_con_return() # Muestra el primer mensaje y guarda 42 en 'valor'
print(f"El valor guardado es {valor}")
print(f"El doble del valor es {valor * 2}")  # Esto solo es posible porque usamos return
```

Concepto integrado: por qué return es indispensable cuando el resultado de una función se necesita para seguir operando con él.

99. Ejercitación propuesta

Todas las funciones de esta ejercitación no llevan parámetros. Los datos que necesiten pedirlos con input() o definirlos como valores fijos dentro del cuerpo de la función.

Nivel Básico

EJ01) Definir una función **saludar()** que imprima "¡Hola, Taller de Programacion I!".

EJ02) Definir una función **mostrar_fecha_actual()** que imprima una fecha fija, por ejemplo "Hoy es 24/07/2026".

EJ03) Definir una función **linea()** que imprima una línea de guiones ("----------") y usarla para separar visualmente dos mensajes distintos.

EJ04) Definir una función **presentacion()** que imprima tu nombre, tu edad y tu ciudad en tres líneas separadas.

EJ05) Definir una función **numero_favorito()** que utilice return para devolver tu número favorito, y luego imprimir ese valor desde fuera de la función.

Nivel Intermedio

EJ06) Definir una función **calcular_area_cuadrado()** que use un lado fijo (por ejemplo, 5) y devuelva el área mediante return.

EJ07) Definir una función **pedir_edad()** que solicite la edad al usuario con input(), la convierta a entero y la devuelva con return.

EJ08) Definir una función **mostrar_tabla_del_2()** que imprima la tabla de multiplicar del 2, del 1 al 10, usando un bucle for dentro de la función.

EJ09) Definir dos funciones, **es_par()** (que devuelva si un número fijo definido dentro de la función es par o impar como texto) y **mostrar_resultado(**) que llame a la primera e imprima el resultado.

EJ10) Definir una función **bienvenida_completa()** que llame internamente a otras tres funciones ya creadas (por ejemplo, **titulo()**, **linea()**, **instrucciones()**), demostrando composición de funciones.

EJ11) Definir una función **calcular_promedio_fijo()** que calcule el promedio de tres números fijos definidos dentro del cuerpo (por ejemplo, 7, 8 y 9) y lo devuelva con return.

EJ12) Definir una función **pedir_dos_numeros()** que solicite dos números por input(), los sume, y devuelva el resultado.

Nivel Avanzado

EJ13) Definir una función **generar_reporte()** que imprima un encabezado, una lista de al menos tres elementos usando un bucle, y un pie de página, todo dentro de la misma función.

EJ14) Definir una función sumar_cuadrados() que calcule la suma de los cuadrados de los números del 1 al 5 (es decir, $1^2 + 2^2 + 3^2 + 4^2 + 5^2$) utilizando un bucle for y devuelva el resultado con return.

(Solo usa multiplicación y acumulación, conceptos ya vistos).

EJ15) Definir una función es_divisible_por_3_y_7() que determine si el número 21 (fijo, definido dentro de la función) es divisible simultáneamente por 3 y por 7, devolviendo True o False según corresponda.

(Usa el operador % y el and lógico, ambos ya vistos en condicionales).

EJ16) Definir una función **simulador_login()** que pida usuario y contraseña por input(), compare con valores fijos definidos dentro de la función (por ejemplo "admin" y "1234"), e imprima si el acceso fue exitoso o no.

EJ17) Definir una función calcular_precio_con_iva() que solicite un precio al usuario con input(), aplique un IVA fijo del 21% definido dentro de la función, y devuelva el precio final con el IVA incluido.

(Usa solo multiplicación y porcentaje, sin necesidad de fórmulas externas).

EJ18) Definir una función calculadora_simple() que, sin usar parámetros, realice lo siguiente:

- Muestre un menú fijo con 3 opciones (sumar, restar, multiplicar).
- Pida dos números al usuario con input().
- Realice una sola operación fija (por ejemplo, la suma) y muestre el resultado en pantalla.
- Para organizar, creá dos funciones auxiliares internas:
    - mostrar_menu() (que imprima el menú).
    - pedir_numeros() (que pida los dos números y los devuelva con return).
- calculadora_simple() debe llamar a estas dos funciones y mostrar el resultado.

EJ19) Reflexión escrita (sin código): explicar con tus palabras la diferencia entre print y return, y dar un ejemplo propio (no visto en la guía) de una situación donde sea imprescindible usar return en lugar de print.

EJ20) Definir una función verificar_multiplo_de_4() que determine si el año 2024 (fijo, definido dentro de la función) es divisible por 4, devolviendo True o False.